SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**

Question Count: 5

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: beffy

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

ANSWER

1. HDFS-Based Storage Solution for a Media Platform

Constraints:

- Data availability > 99.9%
- Replication factor ≤ 3
- Storage growth must be cost-efficient

Design:

- Use HDFS as the primary storage system.
- Set the replication factor to 3 for important media data.

- Distribute data blocks across different DataNodes and racks using rack awareness.
- Use NameNode High Availability (HA) with Active and Standby NameNodes.
- Store frequently accessed media in HDFS, while older/less-used data can be moved to cheaper archival storage.
- Enable compression for metadata, logs, and suitable media-related files to reduce storage requirements.
- Monitor disk utilization and add DataNodes as storage grows.

Justification:
Replication factor 3 provides fault tolerance while remaining within the constraint. Rack-aware placement prevents data loss if an entire rack fails. NameNode HA avoids a single point of failure. Compression and tiered/archival storage keep the solution cost efficient.

2. MapReduce Workflow for Log Processing

Constraint:

Process logs within a 20-minute batch window using limited cluster nodes.

Job Design:

Log Files

↓

Input Splits

↓

Mapper Tasks

↓

Combiner

↓

Shuffle & Sort

↓

Reducer Tasks

↓

Processed Results

Mapper:

- Reads log records in parallel.
- Extracts useful information such as IP address, timestamp, status code, or URL.
- Produces intermediate key-value pairs.

Combiner:

- Performs local aggregation before data is transferred to reducers.
- Reduces network traffic.

Reducer:

- Aggregates mapper results.
- Produces final statistics such as request counts, error counts, or traffic volume.

Task Distribution:

- Divide the log data into multiple input splits.
- Assign mapper tasks to available nodes.
- Use a limited number of reducers based on cluster capacity.
- Prefer data locality, so computation occurs close to where the data is stored.
- Configure appropriate block size and avoid excessive small files.

Justification:
Parallel mappers reduce processing time, while combiners reduce shuffle traffic. Data locality and controlled reducer allocation prevent limited nodes from becoming overloaded, helping the job finish within 20 minutes.

3. YARN Scheduler Policy for ETL, Reporting and ML

Constraints:

- Fairness
- ETL must receive higher priority
- Recovery from node failure

Proposed Policy:

Use Capacity Scheduler with separate queues:

YARN

├── ETL Queue → High Priority

├── Reporting Queue → Medium Priority

└── ML Queue → Normal Priority

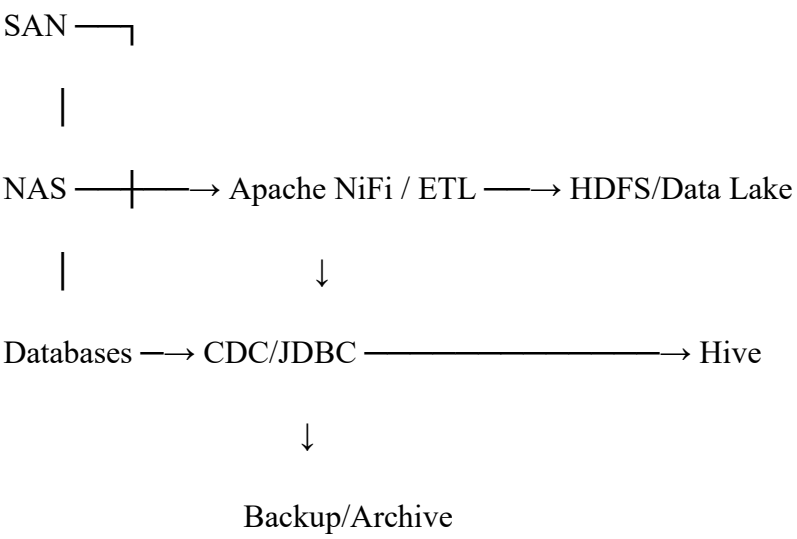
Policy:

- Give the ETL queue higher capacity and priority because ETL is critical.
- Reserve a reasonable amount of cluster resources for reporting and ML to maintain fairness.
- Use resource limits so that one workload cannot consume the entire cluster.
- Enable preemption when high-priority ETL jobs require resources.
- Configure YARN to automatically restart failed containers.
- Use NodeManager health monitoring to detect failed nodes.
- Reschedule failed tasks on healthy nodes.

Justification:
ETL gets priority without completely starving reporting and ML jobs. Queue-based resource allocation provides fairness, while container restart and task rescheduling provide recovery from node failures.

4. Enterprise Data Ingestion from SAN, NAS and Databases

Architecture:



Components:

1. **Apache NiFi**
    - Collects data from different sources.
    - Provides data routing, transformation and monitoring.
2. **SAN/NAS Connectors**
    - Read files from enterprise storage.
    - Transfer only required/new files.
3. **Database Ingestion**
    - Use JDBC or CDC (Change Data Capture).
    - Capture only changed records instead of repeatedly copying the entire database.
4. **HDFS/Data Lake**
    - Central storage for incoming data.
    - Use appropriate replication and directory structures.

5. **Backup**
    - Maintain copies in a separate storage/backup location.
    - Use incremental backups where possible.
6. **Deduplication**
    - Use unique record IDs, checksums/hashes and CDC.
    - Avoid ingesting the same file or record multiple times.

Justification:
NiFi provides controlled ingestion from heterogeneous sources. CDC and checksums minimize duplication, while HDFS and separate backups provide reliable data protection.

5. End-to-End Hadoop Ecosystem Solution

Architecture:

Data Sources

↓

Apache NiFi

↓

Secure Data Transfer

↓

Kafka / Ingestion Layer

↓

HDFS

↓

Hive / Spark

↓

Analytics / ML

↓

Reports / Applications

Design:

- Apache NiFi handles data collection, routing and ETL.
- Use TLS/SSL for secure data movement.
- Use Kerberos authentication to authenticate Hadoop users and services.
- Store data in HDFS with replication factor 3 for fault tolerance.
- Configure NameNode HA using Active and Standby NameNodes.
- Use YARN to manage cluster resources.
- Use Hive for SQL-based analysis.
- Use Spark for large-scale processing and machine learning.
- Use backup/replication mechanisms for critical data.
- Apply access control and auditing to protect sensitive data.

How Constraints Are Satisfied:

Constraint	**Solution**
Secure data movement	TLS/SSL, Kerberos authentication
High availability	NameNode HA, HDFS replication
Data ingestion	Apache NiFi
ETL integration	NiFi + Hive/Spark
Resource management	YARN
Fault recovery	Replication and automatic task recovery
Data security	Authentication, authorization and auditing

Conclusion:

The proposed Hadoop ecosystem provides secure ingestion, highly available storage, efficient ETL, scalable processing, and fault recovery. Apache NiFi acts as the integration layer, while HDFS, YARN, Hive and Spark provide reliable storage, resource management and analytics.